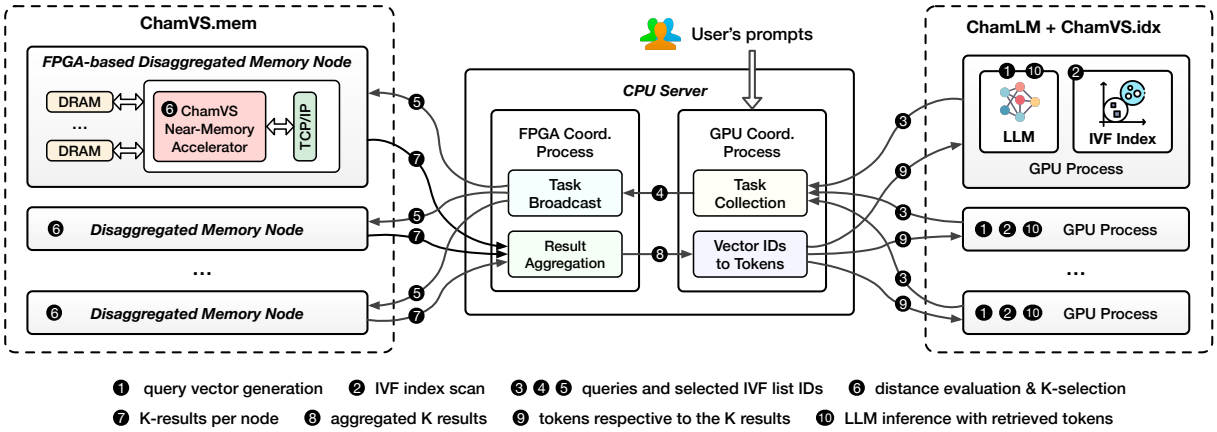

Figure 3: Chameleon is a heterogeneous and disaggregated accelerator system for efficient RALM inference.

assuming 1KB per vector and one thousand vectors per IVF list, a single GB of index can support one million IVF lists, enough for a large database containing one billion vectors. On the other hand, ChamVS.mem is responsible for querying quantized database vectors. ChamVS.mem contains one or multiple disaggregated memory nodes, each with a partition of the database vectors and a near-memory retrieval accelerator prototyped on FPGA for query processing (left side of Figure 3).

Secondly, ChamLM is a multi-GPU LLM inference engine, as shown on the right side of Figure 3. Each GPU, managed by an independent GPU process, can reside on the same or different servers. Currently, ChamLM assigns each GPU a full copy of the LLM, as RALMs can achieve high generation quality even with smaller LLMs [7, 49]. Future larger models could be accommodated by extending ChamLM to support tensor or pipeline parallelism [59, 69, 73] across GPUs. Once a retrieval request is sent, a GPU pauses inference to wait for results. While one potential solution to avoid such GPU idleness is to split the generation into two sub-batches — one executes inference when the other one is waiting for retrieved contents — this approach does not necessarily improve performance. This is because (a) using sub-batches reduces inference throughput, and (b) retrieval latency may not align with inference latency.

Thirdly, the CPU serves as the cluster coordinator, managing the lightweight communication between the GPUs and FPGAs. After receiving search requests from the GPU processes, it dispatches them to the FPGA-based disaggregated memory nodes, aggregates the per-partition results returned by the FPGAs, converts the K nearest neighbor vector IDs into their corresponding texts, and sends the retrieved tokens back to the GPUs. Since each query only requires less than ten KBs of network data transfer, the communication latency is negligible compared to vector search and LLM inference.

Token generation workflow. For each token generation step, the procedure diverges depending on whether the retrieval is invoked. Without retrieval, the GPUs infer the next token as in regular LLMs. With retrieval, the first step is to generate a contextual query vector ❶, either by using the hidden state of the current context [41, 42] or encoding the query tokens through another model [7]. Following this, the IVF index residing on the same GPU is scanned to select the *nprobe* most relevant IVF lists ❷. The query

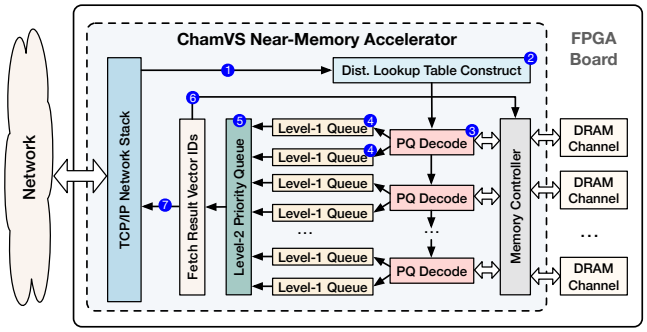

Figure 4: The ChamVS near-memory retrieval accelerator.

vector and the list IDs are then transmitted to the GPU coordinator process running on the CPU node via the network ❸. After recording the association between queries and GPU IDs, the query and list IDs are forwarded to the FPGA coordination process ❹, which broadcasts them to the FPGA-based disaggregated memory nodes ❺. The ChamVS near-memory processor on each node then uses the query vectors to construct distance lookup tables for each IVF list, computes the distances between the query and quantized database vectors, and collects the K nearest neighbors ❻. Subsequently, the result vector IDs and distances from all memory nodes are sent back to the CPU server ❼, which aggregates the results ❽ and returns the tokens of the nearest neighbors to the originating GPU ❾. Finally, the GPU predicts the next token based on both the context and the retrieved tokens ❿.

4 CHAMVS NEAR-MEMORY ACCELERATOR

ChamVS enables high-performance, large-scale vector search by pairing each disaggregated memory node with a near-memory retrieval accelerator. As shown in Figure 4, the accelerator comprises a distance lookup table construction unit, several PQ decoding units for distance evaluations between query vectors and quantized database vectors, a group of systolic priority queues for parallel K-selection, and multiple memory channels.

4.1 PQ Decoding Units

As shown in Figure 4 ❸, each ChamVS accelerator contains multiple PQ decoding units to fully utilize the memory bandwidth. These

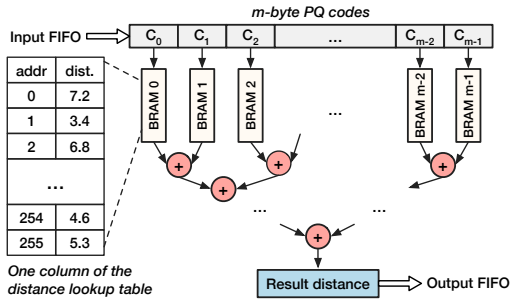

Figure 5: The architecture design of a PQ decoding unit.

units read database vectors (PQ codes) from DRAM and compute their distances to query vectors using a distance lookup table.

The design of a PQ decoding unit involves both operator and pipeline parallelisms, enabling a high throughput of producing one result distance every clock cycle. As shown in Figure 5, the decoding steps — including data ingestion, distance lookups, computation, and output egestion — are fully pipelined, similar to that of [38, 45]. The unit also parallelizes the operators within the distance lookup and computation steps.

Decoding procedure. For each IVF list to scan, the unit first stores the input distance lookup table in BRAM (on-chip SRAM in FPGAs). The shape of the lookup table is $m \times 256$ for the typical 8-bit PQ codes ($2^8 = 256$), where m is the number of bytes per quantized vector. Different table columns are stored in separate BRAM slices, facilitating parallel distance lookups. Subsequently, the PQ codes are loaded from DRAM to the unit via an m-byte-wide FIFO, with each byte serving as an address to retrieve a value from the corresponding column of the table. Finally, an adder tree sums up the retrieved values to produce the approximate distance between the query vector and the quantized database vector.

4.2 Efficient K-Selection Module

The K-Selection module in ChamVS selects the K nearest neighbors from distances computed by the PQ decoding units. Designing an efficient K-selection microarchitecture is challenging, because it has to handle multiple incoming elements per cycle due to the high throughput of PQ decoding units. We propose approximate hierarchical priority queue (AHPQ), a high-throughput, resource-efficient architecture for parallel K-selection in hardware.

4.2.1 Primitive: Systolic Priority Queue. The systolic priority queue facilitates high-throughput input ingestion on hardware accelerators [29, 46], consuming *one input element every two clock cycles.* In short, it is a register array equipped with compare-swap units between the registers, thus *the hardware resource consumption of the queue increases linearly with its length.*

A natural approach to implement K-selection in ChamVS is to instantiate a group of systolic priority queues in a hierarchical structure, as shown in Figure 4 ❹❺. Since a systolic priority queue can only ingest one input every two cycles, two queues, termed as level-one (L1) queues, should be paired with one PQ decoding unit, as it can produce one output per cycle. For each query, each L1 queue collects a subset of the K nearest neighbors, and the level-two (L2) queue subsequently selects the final K results.

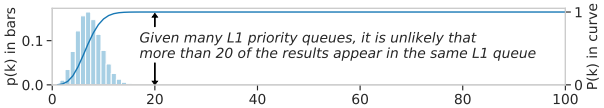

Figure 6: The probability distribution that one out of the 16 L1 priority queues holds k out of the 100 nearest neighbors.

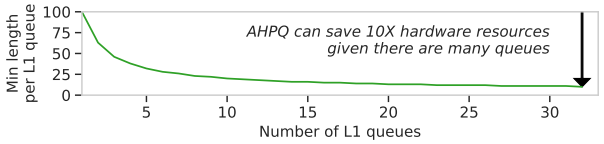

Figure 7: The proposed approximate hierarchical priority queue can save hardware resources by an order of magnitude.

Unfortunately, a straightforward implementation of the hierarchical priority queue can consume excessive hardware resources, making the solution unaffordable even on high-end FPGAs. For example, given 32 instantiated PQ decoding units and $K = 100$, the accelerator would necessitate 64 L1 queues of length 100, an amount that already exceeds the total the total available FPGA resources.

4.2.2 Approximate Hierarchical Priority Queue (AHPQ). **We propose the AHPQ architecture for high-performance and resource-efficient K-selection.** Recognizing that ANN search is inherently approximate, we relax the K-selection objective from selecting the K smallest distances in all queries to collecting precise results in the vast majority of cases, such as in 99% of the queries.

The intuition behind AHPQ is simple: *it is unlikely that all K results are produced by a single PQ decoding unit.* For example, given 16 level-one queues of length K=100, the average number of the results in a queue is only $100/16 = 6.25$. Specifically, the probability that one queue holds k of the K results is $p(k) = C_K^k * (\frac{1}{num_{queue}})^k * (1 - \frac{1}{num_{queue}})^{K-k}$, where C_K^k represents the number of combinations selecting k out of K items. The cumulative probability that a queue contains no more than k of the K results is $P(k) = \sum_{i=0}^{k} p(i)$. Figure 6 shows the probability distribution of p and P given different k in bars and curve: it is almost impossible that a queue holds more than 20 out of the *K=100* results. Thus, the lengths of the L1 queues can be truncated to 20 while producing almost the same results.

Our design aims to reduce the size of the L1 queues while ensuring that the results for 99% of queries remain identical to those obtained with an exact K-selection module. Specifically, for 99% of the queries, none of the L1 queues will omit any result that is supposed to be returned to the user.

Figure 7 shows the resource savings achieved by applying the approximate hierarchical priority queue. As the number of L1 queues increases, the queue sizes can be reduced by an order of magnitude while still retaining 99% of identical results, leading to a corresponding decrease in hardware resource consumption.

4.3 Memory Management and Load Balancing

The memory management mechanism of ChamVS balances workloads across memory nodes and channels. In our current implementation, vectors within each IVF list are evenly partitioned among memory nodes, with these sub-lists further distributed across memory channels to ensure workload balance. For potential scenarios where IVF lists are too small to be partitioned, each list may reside

on different nodes or channels, which could lead to load imbalances, especially with small query batches. Such imbalances can be mitigated by larger batches, as it is less likely that all queries happen to hit the same node or channel. Additionally, for the case of uneven access frequencies across IVF lists, adjusting their placement based on these frequencies can help achieve better load balancing [9].

5 IMPLEMENTATION

Chameleon is implemented in 11K lines of code, including 3K lines of Vitis HLS C/C++ for the ChamVS near-memory accelerator, 1.4K lines of C++ for the CPU coordinator, 3.5K lines of Python for ChamLM, and 3.2K lines of Python for various evaluations. Referring to existing RALM research [41, 42], we build ChamLM on Fairseq [60], a PyTorch-based LLM toolkit. ChamLM extends Fairseq to support multi-GPU inference, initiating retrieval requests, integrating the retrieved tokens into generation processes, and network communication between the retrieval engines and GPU processes. ChamVS.idx uses Faiss [40] for index scanning on GPUs or CPUs. ChamVS.mem integrates an FPGA TCP/IP stack [25]. The CPU coordinator process for query broadcasting and result aggregation is implemented in C++ using the socket library. The simple messages in RALMs allow us to avoid higher-level abstractions like RPCs, minimizing performance overhead.

6 EVALUATION

We evaluate Chameleon to answer the following questions:

- How much performance and energy benefits can ChamVS attain in large-scale vector search? § 6.2
- How does Chameleon perform across different RALMs by introducing heterogeneous accelerators? § 6.3
- Is accelerator disaggregation necessary? § 6.3

6.1 Experimental Setup

LLMs. We evaluate RALM models of similar sizes to those in existing RALM research [7, 31, 51, 72, 86], up to several billions of parameters. We evaluate both smaller (S) and larger (L) decoder-only (Dec) and encoder-decoder (EncDec) models. Table 1 summarizes the four RALMs for evaluation, including input dimensionalities, numbers of layers and attention heads, model sizes, retrieval intervals, and neighbor numbers. For EncDec models, we follow [7] to use a two-layer shallow encoder and a deeper decoder, and set different retrieval intervals. For all the models, we use a vocabulary size of 50K and let them generate 512 tokens per sequence.

Vector datasets. Table 2 summarizes the four evaluated vector datasets. The SIFT and Deep datasets are popular benchmarks for billion-scale ANN. Due to the lack of available datasets for RALM, we create two synthetic datasets by replicating each SIFT vector to the models' dimensionalities (512 and 1024). As a common practice, we set *nlist*, the number of clusters in the IVF index, to approximately the square root of the number of dataset vectors (*nlist=32K*). We set *nprobe* as 32 to scan 0.1% of database vectors per query, for which high recall can be achieved on both real-world datasets (93% on Deep and 94% on SIFT for 100 nearest neighbors). We quantize the SIFT and Deep datasets to 16-byte PQ codes, while the two synthetic datasets adopt 32 and 64-byte PQ codes, respectively.

Table 1: Various RALM configurations in the evaluation.

	Dim.	Layers	Heads	Param.	Interval	K
Dec-S	512	24	8	101M	1	100
Dec-L	1024	96	16	1259M	1	100
EncDec-S	512	2,24	8	158M	8/64/512	10
EncDec-L	1024	2,96	16	1738M	8/64/512	10

Table 2: The vector datasets used in the evaluation.

	Deep	SIFT	SYN-512	SYN-1024
#vec	1E+9	1E+9	1E+9	1E+9
m/D	16/96	16/128	32/512	64/1,024
nprobe/nlist	32/32K	32/32K	32/32K	32/32K
Raw vectors (GB)	384	512	2,048	4,096
PQ and vec IDs (GB)	24	24	40	72

Software. For vector search, we use *Faiss* [1] developed by Meta, known for its optimized PQ implementations for both CPUs and GPUs. Due to its vector-only nature, Faiss's ANN search performance surpasses vector data management systems that support additional relational data functionalities [62]. For LLM inference, we extend Fairseq [60] to support RALMs as introduced in §5.

Hardware. We instantiate the ChamVS near-memory accelerator on AMD Alveo U250 FPGAs (16 nm) equipped with 64 GB of DDR4 memory (4 channels x 16 GB) and set the accelerator frequency to 140 MHz. For a fair comparison, each ChamVS memory node is compared to a CPU-based vector search system with equivalent memory capacity (64 GB) and an 8-core AMD EPYC 7313 processor (7 nm) with a base frequency of 3.0 GHz. We evaluate NVIDIA RTX 3090 GPUs (8nm) with 24 GB GDDR6X memory.

6.2 Large-Scale Vector Search on ChamVS

Search performance. We compare ChamVS with baseline systems using four hardware setups. PQ codes can be processed on CPU/FPGA while the IVF index can be scanned on CPU/GPU, leading to four hardware configurations: CPU, CPU-GPU, FPGA-CPU, and FPGA-GPU. To report the best baseline performance, the CPU and CPU-GPU systems are monolithic, while the FPGA-CPU and FPGA-GPU systems are disaggregated over the network. Figure 8 compares the latency distributions of the four solutions. Each white dot in the violin plots denotes a median latency. The number of CPU cores and the number of accelerators used are listed in the plot legends. We make two primary observations from the experiments:

Firstly, the near-memory accelerator in ChamVS significantly lowers vector search latency. Across different datasets and batch sizes (Figure 8), the FPGA-CPU solution achieves 1.36~6.13× speedup compared to the CPU baseline, and the FPGA-GPU solution shows even higher speedup (2.25~23.72×). This is because the ChamVS near memory accelerator can (a) decode PQ codes in parallel, (b) pipeline the decoding, distance calculation, and K-selection, such that each quantized vector can be processed by the pipeline rapidly.

Secondly, scanning the IVF index on GPU allows further latency improvements compared to the FPGA-CPU solution. As shown in Figure 8, the FPGA-GPU approach achieves 1.04~3.87× speedup compared to the FPGA-CPU solution. This is because the IVF index scan procedure can easily leverage the massively parallelism and the high memory bandwidth of GPUs. In contrast, the hybrid CPU-GPU solution shows little or even negative improvements compared